

ROADMAP DA APROVAÇÃO — ETB-PAY

Dossiê de Revisão de Java Back-End | Turma 3G

1. Mensagem do Tech Lead

Senhores, missão cumprida! Começamos o semestre escrevendo rotinas simples e hoje vocês entregam uma arquitetura bancária funcional. Deixaram de ser apenas "escritores de scripts" e passaram a pensar como verdadeiros Engenheiros de Software.

A curva do Java é íngreme. Ele não te perdoa, não aceita gambiarras e exige rigor estrutural. Vocês dominaram a **Orientação a Objetos**, entendendo que o Encapsulamento é a armadura do negócio. Compreenderam que softwares reais falham e que o **Tratamento de Exceções** é o que separa um sistema amador de um sistema corporativo. E, por fim, cruzaram a ponte da **Persistência com JDBC**, conectando o código de vocês ao banco de dados com segurança máxima.

Este dossiê consolida os 4 Pilares da nossa disciplina. Ele é o seu mapa definitivo para a prova e para as bancas de concurso. Leiam, entendam e executem.

SEÇÃO A: MERGULHO TEÓRICO EXAUSTIVO

Pilar 1: Fundamentos e Tipagem Forte

O Java é uma linguagem **fortemente tipada e de tipagem estática**. Isso significa que toda variável precisa ter seu tipo declarado no nascimento (ex: `int idade;`) e o compilador monitora isso com rigor absoluto. O Java não permite que você guarde um valor decimal (`15.5`) em uma variável inteira acidentalmente, pois isso geraria perda de dados. Se você precisar forçar essa conversão, assumindo o risco, o Java exige a assinatura de um **Casting** explícito (ex: `(int) 15.5`).

Pilar 2: Orientação a Objetos e Encapsulamento

A POO é a alma do Back-End. As **Classes** são as plantas baixas (o molde), enquanto os **Objetos** são as casas reais construídas na memória RAM através do comando `new`. Quando um objeto nasce, ele dispara automaticamente um método especial chamado **Construtor**, usado para garantir que o objeto inicie com os valores vitais preenchidos.

A regra de ouro da arquitetura é o **Encapsulamento**. Nunca declaramos os atributos como `public`. Marcamos o estado interno como `private` para blindá-lo e expomos apenas métodos `public` (Getters e Setters) para controlá-lo. Se o saldo fosse público, qualquer código malicioso poderia alterá-lo para -500. Com o Setter (ou sacar), colocamos um `if` para bloquear o saque negativo.

Pilar 3: Tratamento de Exceções

Um software corporativo interage com a rede, discos e usuários; por isso, erros em tempo de execução **vão acontecer**. Para que o programa não "crashe" abruptamente, usamos a arquitetura: **try** (tente fazer o caminho feliz), **catch** (plano B caso a falha seja capturada) e **finally** (o bloco garantido que sempre executa, ideal para fechar conexões e liberar recursos).

A JVM divide as anomalias em duas árvores críticas: As **Checked Exceptions** (Checadas) são falhas externas incertas, como conectar num banco (**SQLException**). O compilador "checa" e te *obriga* a escrever o try-catch. Já as **Unchecked Exceptions** (Não-cheçadas / Runtime) derivam de erros de lógica do programador (**NullPointerException**) ou digitação do usuário. O compilador não te obriga a tratá-las, mas os bons engenheiros o fazem.

Pilar 4: Persistência (JDBC) e Padrão DAO

A API JDBC é a ponte de conexão do Java. Para organizar a casa, usamos o padrão de projeto **DAO (Data Access Object)**. A regra do DAO é isolar absolutamente toda a lógica de "falar com o banco" em uma classe específica (ex: **ContaDAO**). Assim, suas telas não misturam SQL com HTML ou lógicas de menu.

Para operar, distinguimos DML de DQL: comandos que alteram o banco (INSERT, UPDATE, DELETE) são disparados com **executeUpdate()**, retornando apenas o número de linhas afetadas. Comandos de leitura (SELECT) usam **executeQuery()** e retornam um **ResultSet** contendo os dados. Além disso, sempre usamos a classe **PreparedStatement** (com parâmetros "?") em vez de concatenar strings SQL diretamente, eliminando definitivamente as vulnerabilidades de **SQL Injection**.

SEÇÃO B: ENGENHARIA REVERSA DA BANCA

Q1. O Rigor da Tipagem e Casting

O Erro Clássico: Acreditar que variáveis numéricas no Java se adaptam automaticamente como em Python, aceitando `int x = 10.5;` sem reclamar.

A Regra (Gabarito): O Java é estático e forte. Inserir ponto flutuante num tipo inteiro gera um **Erro de Compilação**, pois a linguagem prevê a perda de dados. O programador deve obrigar a conversão através do Casting: `int x = (int) 10.5;`.

Q2. Encapsulamento: A Falácia da Variável Pública

O Erro Clássico: Dizer que atributos `public` aceleram o código e são indicados para projetos ágeis que precisam de resultados rápidos.

A Regra (Gabarito): Atributos públicos quebram a segurança de estado da classe, permitindo corrupção direta de variáveis vitais. O encapsulamento com `private` e `Setters` é inegociável, pois garante um ponto central para injeção de **Regras de Negócio e Validação**.

Q3. O Comportamento do Bloco Finally

O Erro Clássico: Afirmar nas provas que o bloco `finally` só entra em ação quando uma exceção é lançada e capturada pelo `catch`.

A Regra (Gabarito): O bloco `finally` é uma garantia absoluta da arquitetura Java. **Ele é executado independentemente de ter ocorrido erro ou não.** Por isso, é o local exclusivo e seguro para realizar o fechamento de conexões abertas (`Connection`, `PreparedStatement`), evitando `Connection Leaks` (vazamento de memória).

Q4. A Obrigatoriedade das Exceções (Checked vs Unchecked)

O Erro Clássico: Marcar a opção que diz que o Java te obriga a criar um `try-catch` para proteger cálculos matemáticos (divisão por zero) ou leituras de objetos nulos.

A Regra (Gabarito): Exceções de lógica (`ArithmeticException` ou `NullPointerException`) são subclasses de `RuntimeException` (**Unchecked**). O compilador não exige `try-catch` para elas. A obrigatoriedade do `try-catch` (**Checked**) existe apenas para comunicação externa (Rede, Arquivos, JDBC).

Q5. A Diferença de Execução no JDBC (DML x DQL)

O Erro Clássico: Configurar um comando `INSERT INTO clientes...` e pedir para a aplicação dispará-lo no banco usando o método `executeQuery()`.

A Regra (Gabarito): DML (Insert, Update, Delete) altera o banco sem ler tabelas, portanto exige o método `executeUpdate()`. O `executeQuery()` é exclusivo para `SELECT`, pois a aplicação fica aguardando o envio da matriz de dados (ResultSet) como resposta.

Q6. A Ameaça do SQL Injection

O Erro Clássico: Achar que concatenar strings dinâmicas (`"SELECT * FROM contas WHERE id = " + variavelUsuario`) é uma prática segura desde que se trate de um banco de dados local.

A Regra (Gabarito): Concatenação de variáveis em queries abre a porta para **SQL Injection**, permitindo que o usuário digite comandos SQL maliciosos na tela. O Padrão Ouro de defesa é usar o **PreparedStatement** com marcadores `?`. Ele pré-compila a query, forçando que o valor injetado seja tratado apenas como um dado literal inofensivo.